CLARVIO

Sweeper Pet System Design

STATUS	OWNER	LAST UPDATED
Proposed	To assign	October 16, 2026

Authors	CLARVIO design proposal
Reviewers	Product, engineering, and manager reviewers to assign
Related docs	Project description in project-description/pd.md

Scope	First release workflow, pixel pet interaction, planning, approval, and operations

1. Abstract

CLARVIO turns customer conversations and requirement files into a reviewable delivery plan. This proposal makes Sweeper, a small pixel pet, the guide through intake, clarification, scope confirmation, draft planning, and change requests. The pet helps people find the next useful action; only explicit customer confirmation and manager approval change the plan state.

The first release uses approved company material, text and text based PDF input, hosted language models, mocked employee and historical task data, deterministic scheduling code, and versioned plans. It supports broad requests while flagging work outside known task coverage. Estimates are proposals, and every customer visible commitment requires a manager gate.

2. Goals and Non-Goals

Goals	Non-goals
Evidence linked requirements and useful clarification.	Autonomous customer commitments or assignments.
A readable plan with tasks, owners, effort ranges, and dates.	Guaranteed dates, fixed prices, or contracts.
Manager approval bound to one immutable plan version.	Scanned PDF OCR or a full employee portal.
A friendly pet that signals state accessibly.	Using mock records to claim estimation accuracy.

3. Background and Problem Statement

Teams currently interpret customer requests, resolve gaps, estimate work, and check capacity in separate steps. CLARVIO joins those steps and keeps a trace from source passage or customer answer to requirement, task, owner, and milestone. The interaction should feel welcoming without making a draft date or AI suggestion appear approved.

Sweeper is a presentation layer over an explicit workflow. It carries document scraps into evidence cards, points to high impact questions, rests while a manager reviews, and celebrates only after publication. Each animation maps to a server supplied state and has a text label. The backend owns evidence, planning, approval, and audit records; the pet cannot approve or alter them.

4. Proposed Architecture

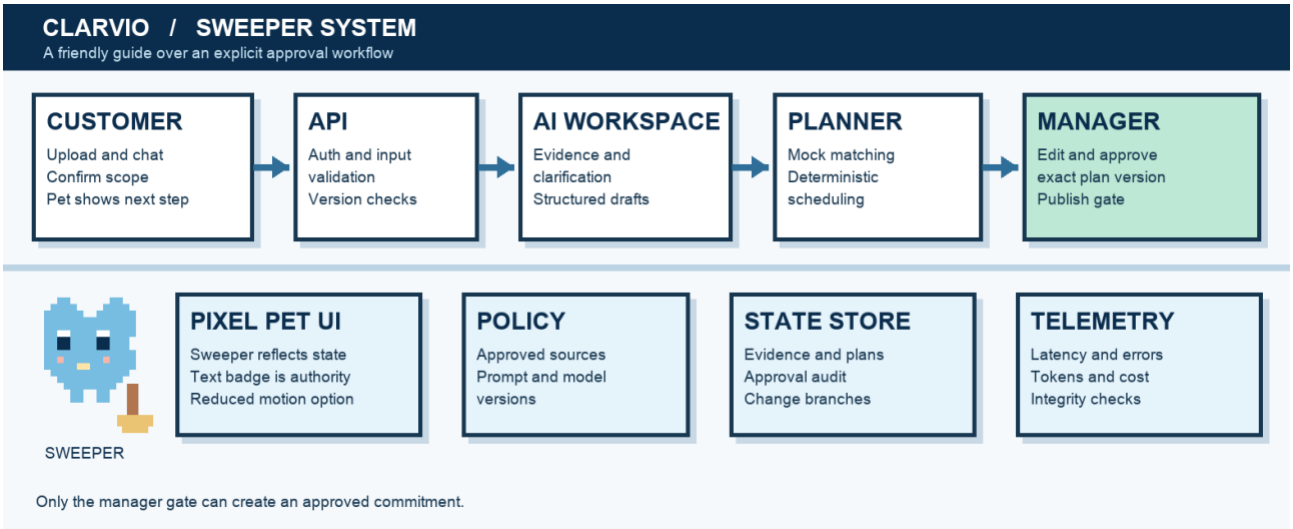


Figure 1. Proposed CLARVIO architecture and Sweeper experience.

Core components

Sweeper is a 16 by 16 sprite with rosy cheeks and a broom tail, scaled in whole numbers. Idle, curious, working, waiting, and celebrate loops use navy, sky, mint, and coral. Text labels and buttons carry meaning; motion can be disabled. Manager review uses quieter styling.

Component	Responsibility	Primary storage	Failure behavior
Pixel web client	Shows Sweeper, evidence cards, plan status, and customer or manager actions.	Server state; local sprite assets	Falls back to text and static art; no state change is lost.
Workspace API	Authenticates actors, validates input, and enforces plan transitions.	Plan and identity stores	Rejects unsafe writes; returns retryable errors for temporary faults.
AI orchestrator	Retrieves approved material and proposes questions, requirements, and tasks.	Source index and run records	Marks draft incomplete and routes to human review after bounded retry.
Matcher and scheduler	Uses mock skills, capacity, dependencies, and calendars to calculate draft dates.	Mock planning records	Reports conflicts and withholds a valid timeline until resolved.
Version and audit store	Persists evidence, plans, approvals, changes, and run provenance.	Database and files	Blocks publish if the approval commit fails.

5. Request Lifecycle

a. A customer starts a workspace, asks a service question, or submits pasted text or a text based PDF. The API creates a request and records file metadata and the customer message.

b. The API authenticates the actor, checks workspace access and file limits, validates the input type, and normalizes text. Unsupported or unreadable files receive an actionable error.

c. The orchestrator loads approved company content, prompt and model versions, confirmed answers, current plan version, and mock skills, workloads, calendars, and task history.

d. The model proposes evidence linked requirements and impact ranked questions. After customer scope confirmation, it proposes tasks and effort ranges; deterministic matching and scheduling produce candidate owners and dates.

e. The service saves the source references, customer decisions, assumptions, run metadata, and a new draft plan version before sending a review notification or publishing a view.

f. Hosted model calls use backend held keys, timeouts, bounded retries, token and cost limits, and a tested second provider. Invalid structured output is retried within budget, then returned for human review.

g. A manager edits and approves one exact version. Publication records approver, time, scope, conditions, and dates. A later customer change creates a separate revision; the approved baseline remains visible. The service emits latency, error, token, cost, fallback, and transition metrics.

6. API and Data Contracts

Primary data contract

Proposed API: POST /workspaces/{id}/inputs and /answers; POST /plans/{id}/confirm-scope, /draft, /approve, and /change-requests. Mutations require an idempotency key and expected plan version.

Field	Type	Required	Description
plan_id	UUID	Yes	Stable plan identity within one customer workspace.
version	Integer	Yes	Monotonic revision used for concurrency and audit.
status	Enum	Yes	Clarifying, Draft, Awaiting review, or Approved.
requirements	Array	Yes	Each item has evidence, acceptance criteria, and confirmation state.
tasks	Array	Yes	Dependencies, owner suggestion, effort range, and rationale.

Field	Type	Required	Description
timeline	Object	No	Milestones, calendar, dates, range, and stated conditions.
approval	Object	No	Approver, time, approved version, and publication record.

PlanVersion is the central contract. Requirements and tasks use stable IDs; every extracted claim points to a source passage or customer message. Dates include calendar and assumptions.

Contract guarantees

a. Validate schema, source links, actor rights, and expected version before a transition. A draft is never published by an AI result alone.
b. Each write has a request ID and idempotency key; versions increase once per accepted transition. Duplicate requests return the recorded result.
c. Store source IDs, model and prompt versions, application version, mock dataset version, scheduler rules, editor, approver, and timestamps with each plan version.
d. The approved PlanVersion is the source of truth for the customer commitment. Pet animation state and proposed estimates are derived views, not authority.

The interface definition should be versioned with the application.

Publish an OpenAPI schema and PlanVersion JSON schema before implementation.

7. Consistency, Idempotency, and Replay

Use optimistic concurrency on plan versions. Approval and publication must be one guarded transition, or publication reads only an approved immutable version. Generated drafts may be retried; approvals are never replayed without manager action. A change request branches from the approved baseline and cannot overwrite it.

Scenario	Expected behavior	Reasoning
Repeated submission	Return stored request result and version.	Idempotency key prevents a second transition.
Database write fails	Reject transition; publish nothing.	Commitment exists only after durable approval record.
Model timeout or invalid output	Retry within budget, then show a reviewable error.	No unsupported draft becomes a commitment.
Config changes mid run	Use captured model, prompt, and rules version.	Run and plan can be reconstructed.

8. Security and Privacy Considerations

a. Customers access only their workspace; managers receive scoped review and approval roles. Every approval checks server side authorization and records the actor.

b. Keep customer documents and model payloads out of routine logs. Store only necessary excerpts for evidence, redact sensitive telemetry, and set input size limits.

c. Store model keys and service credentials in a secret manager. The browser receives no provider key; rotate credentials and restrict access by service role.

d. Debug and replay tools default to read only and cannot publish. Reprocessing creates a new draft linked to the prior version and requires explicit manager review.

e. Define retention, deletion, and data location with product and legal owners before customer launch. Preserve approval audit metadata for the agreed retention period.

9. Operational Readiness

Signal	SLO or alert	Owner	Launch gate
Approval integrity	Zero unauthorized or wrong version publications in gate tests.	Engineering	Required
End to end latency	Measure p95 and set target after baseline load test.	Engineering	Required
Model errors and fallback	Measure invalid output, retry, fallback, and cost per assessment.	AI operations	Required
Version and config drift	Alert when runtime versions differ from recorded run metadata.	Engineering	Required
Recovery and replay	Reconstruct sampled plans and test rollback of prompt or config.	Engineering	Required

Rollout constraint: start with synthetic and labeled cases, then a restricted pilot. Promote only after approval tests, load measurements, and rollback demonstration pass.

10. Alternatives Considered

Alternative	Why it was considered	Why it was not selected
Chat only assistant	Fastest conversational prototype.	Lacks an explicit approval and version boundary.
Pet controls plan state	Feels playful and direct.	Animation cannot be the authority for a commitment.
LLM generated dates	Simple draft generation.	Cannot reliably enforce mock capacity, calendars, and dependencies.
Live HR and project tools	More realistic assignments.	Outside first release and adds integration and data risk.

11. Open Questions

a. Which customer and manager identity system and workspace access model will the first release use?
b. Which company documents and task templates are approved for retrieval, and who owns their review?
c. What retention, deletion, and regional hosting requirements apply to uploaded customer files?
d. Who may edit a manager approved plan, and what approval evidence must appear in the customer view?

12. Decision and Next Steps

Proceed with a narrow prototype in which Sweeper guides clarification and scope confirmation, while the plan state machine enforces manager approval. First build the versioned evidence and plan store; then add model extraction, deterministic scheduling, and the pixel client. Validate against labeled cases and approval boundary tests before a small customer pilot. Broader rollout follows measured service targets and a demonstrated prompt or configuration rollback.

Milestone	Deliverable	Exit criteria
M1	State machine, evidence, versions, and pixel interaction prototype	Scope confirmation and manager approval paths work in test.
M2	Structured extraction, matching, scheduler, and evaluation set	200 or more labeled cases plus scheduling constraints run in CI.
M3	Restricted pilot with telemetry and run provenance	Reviewers can trace every published item and no approval boundary failures occur.
M4	Broader launch with change requests and Markdown export	Measured targets met; rollback tested; product owners approve.